

Transformation d'une application Task Manager

en plateforme SaaS multi-tenant

Documentation technique — Guide étape par étape

Stage de perfectionnement Vue.js Fullstack

Projet Task Manager collaboratif

Sommaire

Sommaire	2
1. Objectif du document	3
1.1 Rappel du contexte de départ	3
1.2 Ce qui change avec le SaaS	3
1.3 Architecture retenue : base de données unique partagée.....	3
2. Étape 1 — Créer la table organizations	4
3. Étape 2 — Ajouter organization_id aux tables existantes	5
4. Étape 3 — Centraliser le contexte d'organisation (middleware).....	7
5. Étape 4 — Forcer le filtre organization_id dans les requêtes.....	8
6. Étape 5 — Adapter le contenu du JWT.....	10
7. Étape 6 — Ajouter l'inscription libre (création d'organisation).....	12
8. Étape 7 — Index pour les performances.....	14
9. Étape 8 — Tests automatisés d'isolation entre organisations.....	15
10. Synthèse méthodologique — pour présentation au superviseur.....	17
10.1 Démarche générale.....	17
10.2 Méthodes techniques utilisées	17
10.3 Points de sécurité à mettre en avant.....	17
10.4 Prochaines étapes (hors périmètre de ce document)	17

1. Objectif du document

Ce document décrit, étape par étape, la démarche technique suivie pour transformer l'application Task Manager (développée durant le stage) en une application SaaS multi-tenant : une seule instance de l'application, une seule base de données, utilisée simultanément par plusieurs entreprises clientes indépendantes.

Chaque section correspond à une étape concrète de l'implémentation, avec le code utilisé et un commentaire explicatif sur chaque ligne. Ce document peut être suivi directement pour réaliser l'implémentation, et sert également de support pour expliquer la démarche et les choix techniques.

1.1 Rappel du contexte de départ

- Application existante : Task Manager, développée en Vue.js 3 (frontend) et Node.js/Express (backend), avec authentification JWT.
- Fonctionnement initial : une seule organisation, les comptes utilisateurs sont créés directement par un administrateur (pas d'inscription libre).
- Rôles existants : admin (gère tout) et utilisateur (accède à ses projets/tâches assignés).
- Sécurité déjà en place : vérification d'appartenance à un projet avant tout accès aux tâches/commentaires (via un helper `isProjectMember`), redirection automatique vers `/login` sur une réponse 403.

1.2 Ce qui change avec le SaaS

Le SaaS doit permettre à plusieurs entreprises (« organisations », aussi appelées « tenants ») d'utiliser la même application sans jamais voir les données des autres. La donnée n'appartient plus seulement à un projet, mais d'abord à une organisation. Toute la démarche ci-dessous consiste à ajouter cette notion d'organisation partout où c'est nécessaire, sans casser ce qui existe déjà.

1.3 Architecture retenue : base de données unique partagée

Il existe plusieurs façons de séparer les données de plusieurs clients dans un SaaS : une base par client, un schéma par client, ou une seule base partagée par tous les clients avec une colonne d'identifiant. C'est cette dernière option (« shared database, shared schema ») qui est retenue ici, car c'est la plus simple à mettre en place et à maintenir pour un projet de cette taille : une seule base de données PostgreSQL, une seule instance de l'API, et chaque ligne de chaque table porte un identifiant d'organisation (`organization_id`) qui indique à quelle entreprise elle appartient.

Note : Le point le plus important de toute la démarche est la vérification systématique de cet `organization_id` à chaque lecture et écriture. C'est ce qui garantit qu'une entreprise ne peut jamais voir ou modifier les données d'une autre, alors qu'elles partagent la même base.

2. Étape 1 — Créer la table organizations

Cette table est le pivot de tout le système multi-tenant : chaque entreprise cliente correspond à une ligne de cette table, et toutes les autres tables vont s'y rattacher.

SQL — création de la table (fichier migrations/001_create_organizations.sql)

```
CREATE TABLE organizations (  
  id SERIAL PRIMARY KEY,  
  -- Identifiant unique auto-incrémenté de l'organisation.  
  
  name VARCHAR(255) NOT NULL,  
  -- Nom affiché de l'entreprise cliente (ex: 'Acme Corp').  
  
  slug VARCHAR(100) UNIQUE NOT NULL,  
  -- Identifiant "lisible" et unique (ex: 'acme-corp'), utile  
  -- si on veut plus tard des URLs du type acme-corp.monapp.com.  
  
  plan VARCHAR(50) DEFAULT 'free',  
  -- Offre souscrite par l'organisation : free, pro, entreprise.  
  -- Sert de base à la facturation (voir section 8).  
  
  status VARCHAR(50) DEFAULT 'trial',  
  -- État du compte : trial (essai), active, suspended, cancelled.  
  -- Permet de bloquer l'accès si l'abonnement n'est plus payé.  
  
  created_at TIMESTAMP DEFAULT NOW()  
  -- Date de création, utile pour le suivi et les statistiques.  
);
```

Chaque ligne de cette table représente une entreprise cliente indépendante. C'est la clé étrangère `organization_id` (ajoutée à l'étape suivante) qui va relier toutes les autres données à cette table.

3. Étape 2 — Ajouter organization_id aux tables existantes

Chaque table qui contient des données propres à une entreprise (projets, tâches, commentaires, utilisateurs, notifications...) doit recevoir une colonne organization_id. C'est cette colonne qui sera vérifiée à chaque requête pour garantir l'isolation des données.

SQL — ajout des colonnes (fichier migrations/002_add_organization_id.sql)

```
ALTER TABLE users
  ADD COLUMN organization_id INTEGER REFERENCES organizations(id);
-- Chaque utilisateur appartient désormais à une seule organisation.
```

```
ALTER TABLE projects
  ADD COLUMN organization_id INTEGER REFERENCES organizations(id);
-- Un projet appartient à l'organisation qui l'a créé.
```

```
ALTER TABLE tasks
  ADD COLUMN organization_id INTEGER REFERENCES organizations(id);
-- Redondant avec le projet parent, mais dupliquer ce champ ici
-- évite d'avoir à faire une jointure vers 'projects' à chaque
-- fois qu'on veut vérifier l'organisation d'une tâche.
```

```
ALTER TABLE comments
  ADD COLUMN organization_id INTEGER REFERENCES organizations(id);
-- Même logique que pour les tâches : évite une jointure.
```

```
ALTER TABLE notifications
  ADD COLUMN organization_id INTEGER REFERENCES organizations(id);
-- Les notifications sont aussi propres à une organisation.
```

Comme l'application contient déjà des données (celles créées pendant le stage), il faut migrer ces données existantes vers une organisation "par défaut" avant de pouvoir rendre la colonne obligatoire.

SQL — migration des données existantes (fichier migrations/003_migrate_existing_data.sql)

```
-- 1. On crée une organisation par défaut pour les données déjà
-- présentes dans la base (celles créées avant le SaaS).
INSERT INTO organizations (name, slug, plan, status)
VALUES ('Organisation initiale', 'org-initiale', 'entreprise', 'active');
-- On récupère l'id généré, ici on suppose que c'est 1
-- (première ligne insérée dans une table vide).

-- 2. On rattache tous les utilisateurs existants à cette organisation.
UPDATE users SET organization_id = 1 WHERE organization_id IS NULL;

-- 3. On rattache tous les projets existants.
UPDATE projects SET organization_id = 1 WHERE organization_id IS NULL;

-- 4. On rattache toutes les tâches existantes.
UPDATE tasks SET organization_id = 1 WHERE organization_id IS NULL;

-- 5. On rattache tous les commentaires existants.
UPDATE comments SET organization_id = 1 WHERE organization_id IS NULL;

-- 6. On rattache toutes les notifications existantes.
UPDATE notifications SET organization_id = 1 WHERE organization_id IS NULL;
```

```
-- 7. Une fois toutes les lignes migrées, on rend la colonne
-- obligatoire pour empêcher toute nouvelle donnée orpheline.
ALTER TABLE users ALTER COLUMN organization_id SET NOT NULL;
ALTER TABLE projects ALTER COLUMN organization_id SET NOT NULL;
ALTER TABLE tasks ALTER COLUMN organization_id SET NOT NULL;
ALTER TABLE comments ALTER COLUMN organization_id SET NOT NULL;
ALTER TABLE notifications ALTER COLUMN organization_id SET NOT NULL;
```

Note : *Cette migration ne s'exécute qu'une seule fois, en production, avant la mise en ligne du SaaS.
Toujours faire une sauvegarde de la base avant de l'exécuter.*

4. Étape 3 — Centraliser le contexte d'organisation (middleware)

Plutôt que de vérifier l'organisation "à la main" dans chaque contrôleur (avec le risque d'oublier un endroit et de créer une fuite de données), on extrait une seule fois l'`organization_id` de l'utilisateur connecté, dans un middleware Express qui s'exécute juste après l'authentification.

JavaScript — middleware/tenantContext.js

```
// Ce middleware s'exécute après le middleware d'authentification JWT.
// Il suppose donc que req.user existe déjà (rempli par ce dernier).
function tenantContext(req, res, next) {

  // On vérifie que l'utilisateur authentifié a bien une organisation.
  if (!req.user || !req.user.organizationId) {
    // Si ce n'est pas le cas, on bloque la requête : sans
    // organisation, on ne peut garantir aucune isolation.
    return res.status(401).json({ error: 'Organisation non identifiée' });
  }

  // On stocke l'organizationId directement sur req, pour que
  // tous les contrôleurs suivants puissent le lire facilement.
  req.organizationId = req.user.organizationId;

  // On passe la main au contrôleur suivant.
  next();
}

module.exports = tenantContext;
```

JavaScript — branchement dans app.js

```
const authenticate = require('./middleware/authenticate');
// Middleware existant : vérifie le JWT et remplit req.user.

const tenantContext = require('./middleware/tenantContext');
// Nouveau middleware créé ci-dessus.

// Toutes les routes protégées passent d'abord par l'authentification,
// puis par l'extraction du contexte d'organisation.
app.use('/api', authenticate, tenantContext, routes);
```

5. Étape 4 — Forcer le filtre organization_id dans les requêtes

La règle à appliquer partout : aucune requête sur une table métier ne doit s'exécuter sans organization_id dans sa clause WHERE. Pour rendre cette règle difficile à oublier, on passe systématiquement par des fonctions "helper" dédiées plutôt que d'écrire du SQL brut dans chaque contrôleur.

JavaScript — services/taskService.js (extrait, avant/après)

```
// AVANT (dans une app mono-tenant) :
// function getTaskById(taskId) {
//   return db.query('SELECT * FROM tasks WHERE id = $1', [taskId]);
// }

// APRES (version multi-tenant) :
function getTaskById(taskId, organizationId) {
  // Le deuxième paramètre organizationId est désormais obligatoire
  // pour toute fonction qui lit ou écrit une donnée métier.
  return db.query(
    'SELECT * FROM tasks WHERE id = $1 AND organization_id = $2',
    // La condition organization_id = $2 est ce qui empêche un
    // utilisateur de l'organisation A de lire une tâche de
    // l'organisation B, même s'il devine ou force son ID.
    [taskId, organizationId]
  );
}

function updateTaskStatus(taskId, organizationId, newStatus) {
  return db.query(
    `UPDATE tasks SET status = $1
    WHERE id = $2 AND organization_id = $3`,
    // Même logique en écriture : impossible de modifier une
    // tâche qui n'appartient pas à sa propre organisation.
    [newStatus, taskId, organizationId]
  );
}

module.exports = { getTaskById, updateTaskStatus };
```

JavaScript — controllers/taskController.js (extrait)

```
const taskService = require('../services/taskService');

async function getTask(req, res) {
  // req.organizationId provient du middleware tenantContext
  // (étape précédente) : on ne fait jamais confiance à une
  // valeur envoyée par le client dans l'URL ou le body pour ça.
  const task = await taskService.getTaskById(
    req.params.id,
    req.organizationId
  );

  if (!task) {
    // Si aucune tâche ne correspond (parce qu'elle n'existe pas,
    // OU parce qu'elle appartient à une autre organisation), on
    // renvoie la même erreur générique : on ne doit jamais
    // révéler qu'une ressource existe chez un autre client.
    return res.status(404).json({ error: 'Tâche introuvable' });
  }
}
```



```
}  
  
    res.json(task);  
}  
  
module.exports = { getTask };
```

Note : Cette même transformation (ajout du paramètre *organizationId* + condition SQL) doit être répétée pour chaque fonction existante : *getProjectById*, *getTasksByProject*, *getCommentsByTask*, *addComment*, etc.

6. Étape 5 — Adapter le contenu du JWT

Le token JWT doit désormais transporter l'identifiant de l'organisation en plus de celui de l'utilisateur, pour que le middleware `tenantContext` puisse le lire à chaque requête.

JavaScript — services/authService.js (génération du token)

```
const jwt = require('jsonwebtoken');

function generateToken(user) {
  // Le payload du token contient maintenant organizationId,
  // en plus des informations déjà présentes (id, rôle).
  const payload = {
    userId: user.id,
    organizationId: user.organization_id,
    // Ajouté : indispensable pour le multi-tenant.
    role: user.role,
  };

  return jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: '24h',
  });
}

module.exports = { generateToken };
```

JavaScript — middleware/authenticate.js (vérification du token)

```
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Token manquant' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Nouvelle vérification : un token sans organizationId est
    // rejeté, il ne peut pas avoir été émis par notre système
    // après la migration vers le SaaS.
    if (!decoded.organizationId) {
      return res.status(401).json({ error: 'Token invalide (organisation manquante)' });
    }

    req.user = decoded;
    // req.user.organizationId sera lu ensuite par tenantContext.
    next();

  } catch (err) {
    return res.status(401).json({ error: 'Token invalide ou expiré' });
  }
}
```

```
module.exports = authenticate;
```

7. Étape 6 — Ajouter l'inscription libre (création d'organisation)

Jusqu'ici, seul un administrateur pouvait créer des comptes. Pour un SaaS public, il faut ajouter un nouveau parcours : une personne s'inscrit seule, ce qui crée automatiquement une nouvelle organisation dont elle devient l'administrateur. Le parcours existant (l'admin crée ensuite les comptes de ses collègues) reste inchangé et continue de fonctionner à l'intérieur de cette organisation.

JavaScript — controllers/signupController.js

```
const db = require('../db');
const bcrypt = require('bcrypt');
const { generateToken } = require('../services/authService');

async function signup(req, res) {
  const { companyName, adminName, email, password } = req.body;

  // On utilise une transaction : soit toutes les insertions
  // réussissent, soit aucune n'est appliquée. Cela évite de se
  // retrouver avec une organisation sans aucun administrateur
  // si une des étapes échoue en cours de route.
  const client = await db.connect();

  try {
    await client.query('BEGIN');

    // 1. Création de l'organisation.
    const orgResult = await client.query(
      `INSERT INTO organizations (name, slug, plan, status)
      VALUES ($1, $2, 'free', 'trial') RETURNING id`,
      [companyName, slugify(companyName)]
    );
    const organizationId = orgResult.rows[0].id;

    // 2. Hachage du mot de passe avant stockage (jamais en clair).
    const passwordHash = await bcrypt.hash(password, 10);

    // 3. Création du premier utilisateur, avec le rôle admin
    // et rattaché à l'organisation qu'on vient de créer.
    const userResult = await client.query(
      `INSERT INTO users (name, email, password_hash, role, organization_id)
      VALUES ($1, $2, $3, 'admin', $4) RETURNING id, organization_id, role`,
      [adminName, email, passwordHash, organizationId]
    );

    // 4. Si tout s'est bien passé, on valide la transaction.
    await client.query('COMMIT');

    // 5. On génère directement un token pour connecter
    // automatiquement l'utilisateur après son inscription.
    const token = generateToken(userResult.rows[0]);
    res.status(201).json({ token });

  } catch (err) {
    // En cas d'erreur, on annule tout (aucune organisation ni
    // utilisateur orphelin ne reste en base).
```

```
    await client.query('ROLLBACK');
    res.status(500).json({ error: "Erreur lors de l'inscription" });

  } finally {
    client.release();
  }
}

module.exports = { signup };
```

Note : La fonction `slugify(companyName)` transforme le nom de l'entreprise en identifiant simple (ex: 'Acme Corp' -> 'acme-corp'), utile pour un futur sous-domaine ou une URL propre.

8. Étape 7 — Index pour les performances

Avec une base partagée entre toutes les entreprises, le volume de données augmente plus vite que dans une application mono-organisation. Comme `organization_id` est présent dans la clause `WHERE` de quasiment toutes les requêtes, il faut l'indexer pour que les recherches restent rapides.

SQL — migrations/004_add_indexes.sql

```
CREATE INDEX idx_projects_org ON projects(organization_id);
-- Accélère toutes les requêtes du type
-- "SELECT * FROM projects WHERE organization_id = ..."

CREATE INDEX idx_tasks_org ON tasks(organization_id);
-- Idem pour les tâches, table la plus consultée de l'application.

CREATE INDEX idx_tasks_org_project ON tasks(organization_id, project_id);
-- Index composite : utile car la plupart des requêtes filtrent
-- à la fois par organisation ET par projet en même temps.

CREATE INDEX idx_comments_org ON comments(organization_id);
CREATE INDEX idx_notifications_org ON notifications(organization_id);
```

9. Étape 8 — Tests automatisés d'isolation entre organisations

Cette étape est indispensable : elle prouve, de façon automatisée et reproductible, qu'une entreprise cliente ne peut jamais accéder aux données d'une autre. C'est aussi un excellent exemple concret à présenter au superviseur pour démontrer la fiabilité de l'isolation multi-tenant.

JavaScript — tests/isolation.test.js (avec Jest + Supertest)

```
const request = require('supertest');
const app = require('../app');

describe('Isolation entre organisations', () => {

  let tokenOrgA, tokenOrgB, taskIdOrgB;

  beforeAll(async () => {
    // On crée deux organisations de test distinctes,
    // chacune avec son propre utilisateur et son propre token.
    tokenOrgA = await getTestToken('orgA@test.com');
    tokenOrgB = await getTestToken('orgB@test.com');

    // On crée une tâche appartenant à l'organisation B.
    const res = await request(app)
      .post('/api/tasks')
      .set('Authorization', `Bearer ${tokenOrgB}`)
      .send({ title: 'Tâche confidentielle organisation B' });

    taskIdOrgB = res.body.id;
  });

  test("un utilisateur de l'organisation A ne peut pas lire une tâche de l'organisation B", async () => {
    const res = await request(app)
      .get(`/api/tasks/${taskIdOrgB}`)
      // On utilise le token de l'organisation A pour tenter
      // d'accéder à une ressource de l'organisation B.
      .set('Authorization', `Bearer ${tokenOrgA}`);

    // On s'attend à un 404 (et non un 403), pour ne même pas
    // révéler que la ressource existe chez un autre client.
    expect(res.status).toBe(404);
  });

  test("un utilisateur de l'organisation A ne peut pas modifier une tâche de l'organisation B", async () => {
    const res = await request(app)
      .patch(`/api/tasks/${taskIdOrgB}/status`)
      .set('Authorization', `Bearer ${tokenOrgA}`)
      .send({ status: 'done' });

    expect(res.status).toBe(404);
  });
});
```

Note : À reproduire pour chaque type de ressource sensible : projets, commentaires, notifications, membres. L'objectif est qu'aucune de ces requêtes croisées ne réussisse jamais, quel que soit l'ID deviné ou forcé côté client.

10. Synthèse méthodologique — pour présentation

Cette section résume la démarche et les méthodes employées, pour servir de support direct en cas de présentation orale ou écrite de la procédure suivie.

10.1 Démarche générale

- Identification du besoin : une seule application, plusieurs entreprises clientes, sans qu'elles se voient entre elles.
- Choix argumenté d'une architecture multi-tenant à base de données partagée (simplicité de mise en œuvre et de maintenance, suffisant pour l'échelle du projet).
- Décomposition en étapes indépendantes et testables : modèle de données, middleware de contexte, requêtes filtrées, authentification, inscription, performance, tests.

10.2 Méthodes techniques utilisées

- Migrations SQL versionnées et numérotées, pour appliquer les changements de schéma de façon reproductible et traçable.
- Middleware Express pour centraliser une information transversale (`organization_id`) plutôt que de la recalculer dans chaque contrôleur.
- Principe de "defense in depth" (défense en profondeur) : la vérification d'organisation est faite à la fois dans le token, le middleware, et chaque requête SQL — un oubli à un niveau n'expose pas les données.
- Transactions SQL (`BEGIN` / `COMMIT` / `ROLLBACK`) pour garantir la cohérence des données lors de la création d'une organisation et de son premier utilisateur.
- Tests automatisés d'isolation, écrits pour prouver — et non seulement affirmer — que la séparation des données entre clients fonctionne.

10.3 Points de sécurité à mettre en avant

- Aucune requête métier n'est exécutée sans filtre `organization_id`.
- Les erreurs d'accès à une ressource d'une autre organisation renvoient un 404 générique, pas un 403 (qui révélerait l'existence de la ressource).
- Le `organization_id` provient toujours du token JWT signé côté serveur, jamais d'une valeur envoyée librement par le client.

10.4 Prochaines étapes (hors périmètre de ce document)

- Mise en place de la facturation par organisation (ex: Stripe), en s'appuyant sur les colonnes `plan` et `status` déjà présentes dans la table `organizations`.
- Déploiement sur une infrastructure de production (base de données managée, monitoring, sauvegardes automatiques).
- Rédaction de la politique de confidentialité et des CGU avant toute ouverture publique du SaaS.